

Reviewer Pack — Minimal Tropical Hodge Dimension and Circuit Depth Inequalit...

Entry e9b8de6819ff · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 21:52:10 UTC

Minimal Tropical Hodge Dimension and Circuit Depth Inequality

Entry ID: e9b8de6819ff **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 21:52:10 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every d -regular graph G , the minimal tropical Hodge dimension ($\text{mhd}(G)$) of its associated Tseitin formula φ_G is linearly correlated with its circuit depth $d(\varphi_G)$, such that $\text{mhd}(G) = \Theta(d(\varphi_G))$.

Rationale (proposer's reasoning):

The connection between tropical geometry and complexity theory can reveal new insights into the structure of computation. Tropical Hodge dimension captures geometric properties of Tseitin formulas, which may correspond to computational resources like circuit depth.

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 70dc1ce87b925553

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that minimal tropical Hodge dimension ($\text{mhd}(G)$) is linearly correlated with circuit depth ($d(\varphi_G)$), support is indicated if, for all 30 seeds, $\text{mhd}(G) \leq 1.5 * d(\varphi_G)$, and falsification occurs if there exists a seed where $\text{mhd}(G) > 2 * d(\varphi_G)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Tropical Hodge Dimension AND Boolean Circuit Depth - Tropical Geometry INTEGRATED WITH Boolean Circuit Complexity - Circuit Depth AND Tseitin Formula IN Tropical Geometry

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1712.05384v2>] Simulation of low-depth quantum circuits as complex undirected graphical models - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1207.1925v1>] Introduction to tropical algebraic geometry - [<http://arxiv.org/abs/1206.1925v1>] Counting Algebraic Curves with Tropical Geometry - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited - [<http://arxiv.org/abs/1406.3065v2>] Lower Bounds for Tropical Circuits and Dynamic Programs - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if n % d != 0:
            return None
        graph = {i: set() for i in range(n)}
        edges = set()
        while len(edges) < (n * d) // 2:
            u, v = random.sample(range(n), 2)
            if u not in graph[v] and v not in graph[u]:
                graph[u].add(v)
                graph[v].add(u)
                edges.add((u, v))
        return graph

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = i
```

```

    for j in range(i+1, n):
        if abs(matrix[j][i]) > abs(matrix[max_row][i]):
            max_row = j
    matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
    factor = matrix[i][i]
    if factor == 0:
        return None
    for j in range(i, n):
        matrix[i][j] /= factor
    for j in range(n):
        if j != i and matrix[j][i] != 0:
            factor = matrix[j][i]
            for k in range(i, n):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def minimal_tropical_hodge_dimension(clauses):
    n = len(clauses)
    matrix = [[Fraction(1 if i == j else 0, 1) for j in range(n)] for i in range(n)]
    for clause in clauses:
        for var in clause:
            matrix[var][var] += Fraction(1, 1)
    reduced_matrix = gaussian_elimination(matrix)
    if reduced_matrix is None:
        return None
    rank = sum(1 for row in reduced_matrix if any(val != 0 for val in row))
    return rank

def circuit_depth(clauses):
    n = len(clauses)
    depth = [0] * n
    for i in range(n):
        for var in clauses[i]:
            depth[var] = max(depth[var], depth[i] + 1)
    return max(depth)

def generate_clauses(graph):
    clauses = []
    for node, neighbors in graph.items():
        clause = [node]
        for neighbor in neighbors:
            clause.append(-neighbor - 1)
        clauses.append(clause)
    return clauses

n_max = 40
instances_tested = 0
total_mhd = 0
total_depth = 0
conjecture_holds = True
counterexample = ""

for n in range(5, n_max + 1, 5):
    for _ in range(6): # Ensure at least 30 instances per seed
        graph = generate_d_regular_graph(n, 2)
        if graph is None:

```

```

        continue
    clauses = generate_clauses(graph)
    mhd = minimal_tropical_hodge_dimension(clauses)
    depth = circuit_depth(clauses)
    if mhd is None or depth is None:
        continue
    instances_tested += 1
    total_mhd += mhd
    total_depth += depth
    if mhd > 2 * depth:
        conjecture_holds = False
        counterexample = f"n={n}, mhd={mhd}, depth={depth}"

if instances_tested == 0:
    return {
        "metric_name": "minimal_tropical_hodge_dimension",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "Graph size must be a multiple of the degree"
    }

mean_mhd = total_mhd / instances_tested
mean_depth = total_depth / instances_tested

return {
    "metric_name": "minimal_tropical_hodge_dimension",
    "metric_value": mean_mhd,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_mhd = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_mhd} std=0.0 support_fraction=1.0")
    elif any(not r["conjecture_holds"] and "counterexample" in r for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
hodge_dimension', 'metric_value': 25.0, 'instances_tested': 24, 'n_max': 40, 'conjecture_holds': Fal
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
TRIAL: {'metric_name': 'minimal_tropical_hodge_dimension', 'metric_value': 25.0, 'instances_tested'
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_56998384.py", line 145, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~
```

File "/home/ludo/Scrivania/

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed, the minimal tropical Hodge dimension (mhd(G)) exceeds twice the circuit depth ($d(\phi_G)$), which co | next: Investigate further to identify the specific conditions under which this counterexample occurs and explore potential reasons for the discrepancy.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13354
2	preregistration	ollama_remote	glm4:latest	0	0	9516
3	novelty	ollama_remote	glm4:latest	0	0	8194
4	novelty	ollama_remote	glm4:latest	0	0	9795
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15067
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12420
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14725
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15459
9	judge	ollama_remote	glm4:latest	0	0	11838

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110369 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e9b8de6819ff.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e9b8de6819ff.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e9b8de6819ff.tar.gz (if generated)